

Checklist

Platform Team Organization

1. Platform Team Foundation

- ☐ Treat the platform as a **product**, not a one-time IT project.
- ☐ Define the platform's vision and value proposition.
- ☐ Identify the internal customers the platform serves.
- ☐ Create a cross-functional platform team rather than organizing purely around technical silos.
- ☐ Make the team responsible for continuously evolving the platform.
- ☐ Ensure the platform team enables development teams rather than becoming another ticket-processing function.

2. Leadership & Roles

- ☐ Assign an accountable platform leader responsible for value, resources, stakeholders, and overall direction.
- ☐ Establish ownership of the technology strategy.
- ☐ Establish ownership of the product roadmap.
- ☐ Establish ownership of engineering delivery.
- ☐ Establish ownership of marketing, communication, and platform adoption.
- ☐ Establish responsibility for support and professional services.
- ☐ Remember that **roles do not have to be filled by individual people**; one person may cover several roles in a small team.
- ☐ Revisit and split responsibilities as the platform team grows.

The book frames these responsibilities similarly to running a small product company, with CEO-, CTO-, product-, engineering-, marketing-, and support-style functions.

3. Technology Strategy

- ☐ Decide which base technologies the platform will use.
- ☐ Determine whether portability across base platforms is required.

- ☐ Plan how the platform can evolve without creating large migration efforts.
- ☐ Decide where open-source components fit.
- ☐ Decide what should be built internally versus what should be purchased, outsourced, or assembled.
- ☐ Document important technical decisions.
- ☐ Make the engineering team's quality and expertise visible to help attract talent.

4. Product & Delivery

- ☐ Build the roadmap from both customer input and the overall platform vision.
- ☐ Prioritize work rather than accepting every feature request.
- ☐ Coordinate product priorities with engineering capacity and delivery timelines.
- ☐ Account for technical complexity and available skills when planning delivery.
- ☐ Maintain a balanced roadmap rather than letting individual customers dictate the platform's direction.

5. Internal Marketing & Adoption

- ☐ Give the platform a recognizable name and identity.
- ☐ Clearly communicate its value proposition.
- ☐ Publish regular updates.
- ☐ Maintain an internal online presence.
- ☐ Hold tech talks, community meetings, hackathons, or other internal events.
- ☐ Create channels where users can ask questions and provide feedback.
- ☐ Prefer earning voluntary adoption through value rather than relying only on mandates.
- ☐ Continuously listen to users and respond to changing needs.

Platform adoption is presented as a two-way conversation: teams should listen to customers while still protecting the product vision and roadmap.

6. East-West Team Alignment

- ☐ Break down silos between infrastructure, operations, networking, security, storage, compute, and related functions.
- ☐ Establish common outcomes across participating teams.
- ☐ Avoid exposing internal organizational boundaries through the platform experience.

- ☐ Provide users with cohesive interfaces rather than forcing them to navigate across multiple teams.
- ☐ Focus discussions on **what outcome is needed** before debating **how it should be implemented**.
- ☐ Preserve team autonomy where possible.
- ☐ Where collaboration is impossible, decide whether to **fix, wrap, or work around** the dependency.

The goal is to avoid “shipping the org chart” into the platform and instead create aligned goals and fluid communication across infrastructure and operations.

7. North-South Team Alignment

- ☐ Establish transparent communication between platform builders and platform consumers.
- ☐ Understand development teams’ desire for speed and flexibility.
- ☐ Understand operations teams’ concerns about stability, security, and compliance.
- ☐ Avoid “us versus them” relationships.
- ☐ Focus discussions on customer outcomes and organizational value.
- ☐ Treat platform adoption as an ongoing relationship rather than a one-time rollout.

8. Customer Needs

- ☐ Identify problems before jumping to solutions.
- ☐ Observe actual developer workflows.
- ☐ Investigate bottlenecks and impediments.
- ☐ Use techniques such as the **5 Whys** or value stream mapping to identify root causes.
- ☐ Avoid blindly copying platforms or tools used by other organizations.
- ☐ Validate proposed features against actual organizational needs.
- ☐ Treat recurring user difficulties as possible platform-design problems.

9. Customer Personas

- ☐ Identify developers as a primary customer group.
- ☐ Identify the project or account administrators.

- ☐ Identify system administrators and operational users.
- ☐ Consider the end users of applications running on the platform.
- ☐ Determine the different support needs of each persona.
- ☐ Design platform interfaces and services with these distinct personas in mind.

10. Customer Engagement Model

- ☐ Provide effective **self-service** capabilities.
- ☐ Make APIs and interfaces consistent and predictable.
- ☐ Provide onboarding and setup assistance where needed.
- ☐ Automate repetitive setup tasks rather than performing them manually every time.
- ☐ Offer consulting for unusual or complex problems when appropriate.
- ☐ Avoid allowing consulting work to consume so much capacity that the platform team becomes a professional-services organization.
- ☐ Create and support a user community.
- ☐ Use co-creation with customers for suitable platform features.
- ☐ Periodically rebalance the effort spent on self-service, setup, consulting, community, and co-creation.

Communities can both scale support and provide valuable feedback; repeated questions can reveal weaknesses in platform design.

11. Support & Enablement

- ☐ Provide users with documentation and a knowledge base.
- ☐ Provide training and onboarding where required.
- ☐ Ensure support work is recognized when engineers perform it.
- ☐ Track how much engineering time is consumed by support.
- ☐ Automate or redesign areas generating repetitive support requests.
- ☐ Consider creating a network of internal architects, champions, or ambassadors.
- ☐ Use enablement to increase teams' autonomy rather than providing permanent staff augmentation.

12. Skills & Cognitive Load

- ☐ Regularly assess the engineering skills available across teams.

- ☐ Develop a broad population of capable senior engineers rather than depending on a small number of experts.
- ☐ Continuously evolve skills as technologies and methodologies change.
- ☐ Identify outdated mental models that make new technologies harder to understand.
- ☐ Teach principles and new ways of thinking—not only APIs and tools.
- ☐ Help development teams handle the cognitive load that remains after platform abstractions are introduced.

Enabling teams should increase the autonomy of development teams, while platform teams reduce the complexity of underlying systems.

13. Decide Whether You Actually Need a Platform

- ☐ Confirm that building a platform is necessary before creating one.
- ☐ Determine whether existing base platforms already provide sufficient capabilities.
- ☐ Consider whether enablement and skills development could solve the problem instead.
- ☐ Consider a **Thinnest Viable Platform** rather than a large custom platform.
- ☐ Build only capabilities that are genuinely unique or valuable to the organization.
- ☐ Spend time observing development teams before committing to major platform investments.
- ☐ Keep expert teams free enough to respond to emerging technologies and paradigm shifts.

A team can sometimes reduce cognitive load and improve engineering capability without building or maintaining a platform.

14. Recruiting & Retention

- ☐ Make recruiting a leadership responsibility.
- ☐ Use senior leaders for important candidate outreach where useful.
- ☐ Promote the organization's genuine strengths rather than copying perks from technology companies.
- ☐ Make interesting technical work visible internally and externally.
- ☐ Pay people sufficiently that compensation is not a persistent concern.
- ☐ Provide learning and career-development opportunities.

- ☐ Give team members visibility through conferences, talks, articles, or leadership interactions.
- ☐ Build strong professional relationships that can outlast individual roles or employment.

Final Platform Health Check

- ☐ Are customers choosing the platform because it provides value?
- ☐ Does the platform reduce developer cognitive load?
- ☐ Can developers complete common tasks through self-service?
- ☐ Do infrastructure and operations teams collaborate effectively?
- ☐ Is customer feedback regularly influencing the roadmap?
- ☐ Are recurring support problems being turned into product improvements?
- ☐ Is the platform team operating like a product organization rather than a ticket queue?
- ☐ Are responsibilities clear even when individuals perform multiple roles?
- ☐ Are the organization's engineering skills improving?
- ☐ Is the platform still the simplest and most valuable solution to the problem?